Maven & Gradle — Build Tools

Java Backend Interview — Short Notes

1. Overview & Comparison

Dimension	Maven	Gradle
Config format	XML (pom.xml) — verbose but explicit.	Groovy DSL (build.gradle) or Kotlin DSL (build.gradle.kts) — concise.
Build model	Convention over configuration. Fixed lifecycle phases.	Task graph (DAG). Fully programmable. Custom tasks easy.
Performance	No incremental build by default. Full rebuild common.	Incremental build, build cache, parallel execution — significantly faster.
Dependency mgmt	pom.xml with <dependency> blocks. BOMs via dependencyManagement.	dependencies block. Platforms (BOM). Version catalogs (libs.versions.toml).
Plugin system	Mature ecosystem. Spring Boot, Surefire, Checkstyle, SpotBugs.	Same plugins + Gradle-specific. Android only supports Gradle.
Multi-module	<modules> in parent POM. Aggregator pattern.	settings.gradle includes subprojects. More flexible task sharing.
Spring Boot	spring-boot-maven-plugin. mvn spring-boot:run.	org.springframework.boot plugin. gradle bootRun.
Learning curve	Easier to start — conventions clear.	Steeper — DSL + task model. Kotlin DSL adds complexity.
Industry usage	Dominant in enterprise Java. Most tutorials use Maven.	Preferred for Android, newer Java projects, large monorepos.

2. Maven Deep Dive

2.1 Build Lifecycle

- Maven has 3 built-in lifecycles: **default** (build), **clean** (cleanup), **site** (documentation).
- Each lifecycle has phases. Running a phase runs all preceding phases.

Phase	Description	Bound Plugin Goal
validate	Validate project structure and config.	—
compile	Compile source code to target/classes.	maven-compiler-plugin:compile
test-compile	Compile test source code.	maven-compiler-plugin:testCompile
test	Run unit tests (JUnit/TestNG). Fail if tests fail.	maven-surefire-plugin:test
package	Package compiled code into JAR/WAR.	maven-jar-plugin:jar
verify	Run integration tests. Check quality gates.	maven-failsafe-plugin:integration-test
install	Install artifact into local ~/.m2 repository.	maven-install-plugin:install
deploy	Deploy artifact to remote repository (Nexus/Artifactory).	maven-deploy-plugin:deploy

```
# Common Maven commands
mvn clean install # clean + full build + install to local repo
mvn clean package # clean + compile + test + package
```

```
mvn test # run unit tests only
mvn verify # run unit + integration tests
mvn spring-boot:run # run Spring Boot app
mvn clean install -DskipTests # skip tests
mvn clean install -Dmaven.test.skip=true # skip compilation too
mvn dependency:tree # show full dependency tree
mvn dependency:analyze # find unused/undeclared deps
mvn versions:display-dependency-updates # check for newer versions
```

2.2 POM Structure

```
<project xmlns='http://maven.apache.org/POM/4.0.0'>
<modelVersion>4.0.0</modelVersion>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>3.2.0</version>
</parent>
<groupId>com.example</groupId>
<artifactId>order-service</artifactId>
<version>1.0.0-SNAPSHOT</version>
<packaging>jar</packaging>
<properties>
<java.version>21</java.version>
<mapstruct.version>1.5.5.Final</mapstruct.version>
</properties>
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
<!-- version inherited from parent BOM -->
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>
</dependencies>
</project>
```

2.3 Dependency Scopes

Scope	Compile	Test	Runtime	Description
compile	Yes	Yes	Yes	Default. On all classpaths.
test	No	Yes	No	Only for test compilation and execution. JUnit, Mockito, Testcontainers.
runtime	No	Yes	Yes	Not needed at compile time. JDBC drivers, logging impl (logback).
provided	Yes	Yes	No	Provided by container/runtime. Servlet API in WAR — Tomcat provides it.
system	Yes	Yes	No	Like provided but explicit path. Avoid — not portable.

import	N/A	N/A	N/A	Only in dependencyManagement for BOM import. Pulls in dependency versions.

2.4 Multi-Module Maven Project

```
# Parent POM (aggregator)
<modules>
<module>order-api</module>
<module>order-service</module>
<module>order-repository</module>
</modules>
<packaging>pom</packaging>
# Child module references parent
<parent>
<groupId>com.example</groupId>
<artifactId>parent-pom</artifactId>
<version>1.0.0-SNAPSHOT</version>
</parent>
```

3. Gradle Deep Dive

3.1 Task Graph & Build Phases

- **Initialization**: reads settings.gradle, determines which projects to build.
- **Configuration**: evaluates build scripts, builds task graph (DAG of all tasks).
- **Execution**: runs selected tasks in dependency order.

```
# Common Gradle commands
./gradlew build # compile + test + package
./gradlew clean build # clean then build
./gradlew test # run unit tests
./gradlew bootRun # Spring Boot run
./gradlew bootJar # Spring Boot executable JAR
./gradlew dependencies # show dependency tree
./gradlew tasks # list all available tasks
./gradlew build --scan # build scan for performance analysis
./gradlew test --tests '*OrderServiceTest*' # run specific test
./gradlew build -x test # skip tests
```

3.2 build.gradle (Kotlin DSL)

```
plugins {
id("org.springframework.boot") version "3.2.0"
id("io.spring.dependency-management") version "1.1.4"
kotlin("jvm") version "1.9.21"
id("jacoco")
}
group = "com.example"
version = "1.0.0-SNAPSHOT"
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }
dependencies {
implementation("org.springframework.boot:spring-boot-starter-web")
implementation("org.springframework.boot:spring-boot-starter-data-jpa")
runtimeOnly("org.postgresql:postgresql")
```

```
testImplementation("org.springframework.boot:spring-boot-starter-test")
testImplementation("org.testcontainers:postgresql")
}
tasks.test {
useJUnitPlatform()
finalizedBy(tasks.jacocoTestReport)
}
tasks.jacocoTestReport {
reports { xml.required = true }
}
```

3.3 Gradle Dependency Configurations

Configuration	Maven Equivalent	Description
implementation	compile	Compile + runtime. NOT exposed to dependents (better encapsulation).
api	compile (transitive)	Exposed to dependents — transitive dependency. Use sparingly in libraries.
testImplementation	test	Test compile + runtime only.
runtimeOnly	runtime	Runtime only. Not on compile classpath.
compileOnly	provided	Compile only. Not in runtime JAR. Lombok, annotation processors.
testRuntimeOnly	—	Test runtime only. DB drivers for integration tests.
annotationProcessor	—	Annotation processors. Lombok, MapStruct, Dagger.

3.4 Incremental Build & Build Cache

- **Incremental build**: Gradle tracks task inputs/outputs. If unchanged, task is UP-TO-DATE. Skip re-running.
- **Build cache**: cache task outputs by input hash. Shared across machines (remote build cache).
- **Configuration cache**: cache build configuration phase. Speeds up repeat builds significantly.

```
# Enable remote build cache in gradle.properties
org.gradle.caching=true
org.gradle.parallel=true
org.gradle.configureondemand=true
org.gradle.jvmargs=-Xmx4g -XX:+UseParallelGC
```

4. Artifact Repository Management

Repository	Type	Description	Use Case
Maven Central	Public	Primary public Java artifact repository.	Download open-source dependencies.
Nexus Repository	Self-hosted	Sonatype Nexus — proxy + hosted + group repos.	Enterprise artifact management, caching external deps, publishing internal artifacts.
JFrog Artifactory	Self-hosted / SaaS	Universal artifact repository — Maven, npm, Docker, etc.	Enterprise, polyglot artifact management, fine-grained access control.
GitHub Packages	Cloud	Host Maven packages in GitHub org.	Open source projects or GitHub-first teams.

Local ~/.m2	Local cache	Maven caches downloaded artifacts locally.	Offline builds, install local artifacts with mvn install.

5. Interview Questions

Maven

Q1. What is the difference between mvn install and mvn deploy?

→ mvn install: compile, test, package, then install artifact to LOCAL ~/.m2 repository — available to other local projects. mvn deploy: does all of install PLUS uploads artifact to REMOTE repository (Nexus/Artifactory) — available to team. Use install for local development, deploy in CI/CD pipelines.

Q2. What is a BOM (Bill of Materials) in Maven?

→ POM with packaging=pom that only contains dependencyManagement — no actual code. Imported into your pom via scope=import. Provides consistent, tested dependency version set. Spring Boot parent is a BOM. Use when you want Spring's curated versions: import spring-boot-dependencies BOM and omit versions from individual dependencies.

Q3. How do you resolve dependency conflicts in Maven?

→ Maven uses nearest-wins strategy: the dependency closest to your project in the dependency tree wins. Diagnose with mvn dependency:tree. Fix by explicitly declaring the version you want in your pom.xml (nearer = wins). Or use dependencyManagement to pin version. Maven does NOT support multiple versions of same artifact.

Q4. What is the difference between SNAPSHOT and RELEASE versions?

→ SNAPSHOT (e.g., 1.0.0-SNAPSHOT): under development. Maven re-downloads from repo each time — always gets latest. RELEASE (e.g., 1.0.0): immutable. Once published, never changes. Use SNAPSHOT during development, RELEASE for production deployments. CI/CD pipelines publish SNAPSHOT on each build, RELEASE on tag/version bump.

Gradle

Q5. Why is Gradle faster than Maven?

→ Gradle: (1) Incremental build — skips tasks whose inputs/outputs unchanged. (2) Build cache — reuses outputs from previous builds or other machines. (3) Parallel task execution — independent tasks run concurrently. (4) Configuration cache — skips configuration phase on repeat runs. Maven runs all phases sequentially every time.

Q6. What is the difference between implementation and api in Gradle?

→ implementation: dependency not exposed to modules that depend on yours — better encapsulation, faster compile (dependents don't recompile when implementation dep changes). api: exposed to dependents — they can use classes from your api dependencies directly. Use implementation by default; api only for library modules that intentionally expose transitive types.

Q7. How does Gradle handle multi-module projects?

→ settings.gradle.kts: include("order-api", "order-service", "order-repo"). Each subproject has its own build.gradle.kts. Root build.gradle.kts can apply common configuration to all subprojects. subprojects { apply(plugin = "java") }. Inter-module deps: implementation(project(":order-api")). Gradle builds only changed modules — incremental.